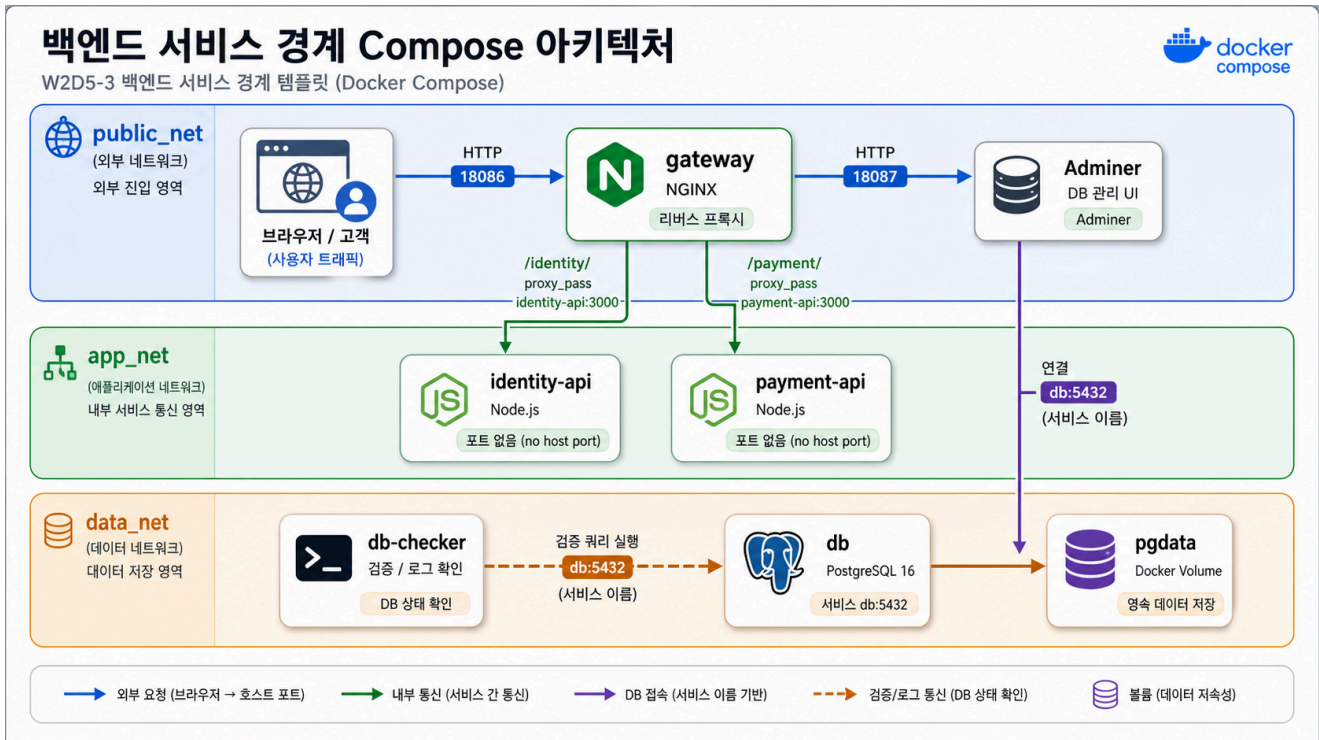


3교시: 당근형 백엔드 서비스 경계 template



수업 목표

- gateway 뒤에 identity API와 payment API를 분리한다.
- 공통 서비스가 많아질 때 service boundary와 장애 영향 범위를 설명한다.
- Adminer가 db service name으로 PostgreSQL에 접속하는 흐름을 확인한다.

언제 쓰는가

W1D4의 당근형 백엔드 경계 사례를 Compose로 줄인다. 사용자/신뢰 정보는 identity API, 결제 흐름은 payment API로 나뉘고, gateway가 외부 진입점을 맡는다. DB 관리 UI는 확인 도구로 붙이되 공개 범위를 항상 의식한다.

Template

```
cd week2/day5/labs/compose-architectures/02-web-postgres-admin
docker compose config
docker compose up -d
docker compose ps
```

compose.yaml 읽기

gateway가 외부 진입점을 맡고, identity/payment API는 내부 service로 남는 점을 코드에서 확인한다.

```
services:
  gateway:
    image: nginx:1.27-alpine
    ports:
      - "18086:80"                # 외부 공개 entypoint는 gateway 하나로 모은
다.
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
                                     # /identity/, /payment/ routing 규칙을 nginx
에 주입한다.
    depends_on:
      - identity-api
      - payment-api
    networks:
      - public_net                # 외부 요청을 받는 gateway 영역
      - app_net                   # 내부 API로 routing하는 영역

  identity-api:
    image: node:20-alpine
    command: ["node", "server.js"]
    volumes:
      - ./apps/identity-api:/app:ro
    environment:
      PORT: 3000                  # host에는 공개하지 않고 gateway가 내부 port로
호출한다.
    networks:
      - app_net

  payment-api:
    image: node:20-alpine
    command: ["node", "server.js"]
    volumes:
      - ./apps/payment-api:/app:ro
    environment:
      PORT: 3000
    networks:
```

```

- app_net

adminer:
  image: adminer:4
  ports:
    - "18087:8080"          # DB 확인 도구이므로 실습 후 노출을 정리해야 한다.
  depends_on:
    - db
  networks:
    - public_net            # 실습용 UI는 host에 공개된다.
    - data_net              # DB에는 data 영역으로 접근한다.

db:
  image: postgres:16
  networks:
    - data_net              # DB는 gateway/app 영역과 분리한다.

networks:
  public_net:
  app_net:
  data_net:

```

여기서 중요한 포인트는 identity-api, payment-api에 ports가 없다는 점이다. 사용자는 gateway로만 들어오고, 내부 API는 Compose network 안에서 service name으로만 호출된다.

구성:

Service	역할	공개 범위
gateway	static page, API route	host 18086
identity-api	user/trust API	Compose network 내부
payment-api	payment mock API	Compose network 내부
db	PostgreSQL 16	Compose network 내부
adminer	DB 관리 UI	host 18087
db-checker	DB 연결 확인 app	logs로 결과 확인

트래픽/부하 성향 노트

백엔드 경계 구조에서는 모든 외부 요청이 gateway를 먼저 통과한다. 하지만 실제 CPU 병목은 gateway보다 내부 API의 business logic에서 생기는 경우가 많다.

Service	트래픽 성향	CPU 부하	메모리/상태 부하	운영에서 먼저 볼 것
gateway	route별 요청이 모두 진입	routing/TLS/logging이 많으면 증가	connection buffer	route별 4xx/5xx, upstream error
identity-api	로그인/사용자 조회 traffic	token 검증, 권한 계산에서 증가	session/cache 연동 시 증가	auth 실패율, latency
payment-api	결제/정산 요청 traffic	validation, 외부 PG 연동 처리에서 증가	retry queue나 idempotency store	중복 요청, timeout
db	user/payment metadata 저장	index 없는 조회, transaction에서 증가	connection, buffer/cache	lock, slow query
adminer	운영 traffic이 아니라 관리 traffic	낮음	session	수업 후 노출 정리

이 구조의 핵심은 “gateway가 바쁘다”와 “business API가 무겁다”를 분리하는 것이다. gateway log는 입구 증거이고, 실제 원인은 identity/payment API나 DB에 있을 수 있다.

Check

```
curl -I http://localhost:18086
curl -I http://localhost:18087
curl -s http://localhost:18086/identity/users
curl -s http://localhost:18086/payment/payments
docker compose logs db-checker --tail 30
```

Adminer login:

항목	값
System	PostgreSQL
Server	db
Username	postgres
Password	postgres
Database	app

실무 해석

gateway는 /identity/를 identity-api:3000으로, /payment/를 payment-api:3000으로 보낸다. Adminer에서 server를 localhost로 넣으면 실패한다. Adminer container 입장에서 localhost는 자기 자신이고, DB service는 Compose DNS 이름 db다.

Cleanup

```
docker compose down  
# DB를 초기화할 때만  
# docker compose down -v
```